

Portfolio Implementation Plan

Exhaustive Phase-by-Phase Engineering Guide

Based on PRD/TRD v3.0 — Unified Project Model

How to read this document: Every task below is atomic — one thing, one outcome. Work top to bottom within each phase. A  marker means "definition of done." Never move to the next task until the current one passes its . Time estimates are in parentheses. Total: ~116 hours across all phases.

PRE-PHASE: Environment Setup (~3 hours)

TASK 0.1 — Create Accounts & Projects

Time: 30 min

1. Create a new GitHub repo: `portfolio` — private, initialize with README, `.gitignore: Node`
2. Go to firebase.google.com → New Project → name: `portfolio-prod`
 - Enable Google Analytics: No
 - After creation: Enable **Firestore** (Native mode, region: `us-central1`)
 - Enable **Authentication** → Sign-in method → Email/Password → Enable
 - Enable **Storage** → Default bucket → Start in production mode
3. Go to vercel.com → New Project → import GitHub repo
 - Framework preset: Next.js
 - Root directory: `.`
 - Do NOT deploy yet — just link the repo

 Firebase project exists with Firestore, Auth, Storage active. Vercel project linked to repo.

TASK 0.2 — Bootstrap Next.js App

Time: 20 min

```
bash
```

```
npx create-next-app@latest portfolio \
  --typescript \
  --tailwind \
  --eslint \
  --app \
  --src-dir=no \
  --import-alias="@/*"

cd portfolio
```

Install all dependencies upfront:

```
bash

# Firebase
npm install firebase firebase-admin

# Forms & Validation
npm install react-hook-form zod @hookform/resolvers

# Markdown
npm install react-markdown remark-gfm rehype-highlight rehype-raw
npm install @uiw/react-md-editor

# Animation
npm install framer-motion

# Icons
npm install lucide-react

# Email
npm install @emailjs/browser

# Analytics
npm install @vercel/analytics

# Dev tooling
npm install -D husky lint-staged prettier @types/node
```

✔ `npm run dev` runs on localhost:3000 with no errors.

Time: 20 min

1. Firebase Console → Project Settings → General → "Your apps" → Add app → Web
 - App nickname: `portfolio-web`
 - Copy the `firebaseConfig` object — these are your `NEXT_PUBLIC_FIREBASE_*` values
2. Firebase Console → Project Settings → Service accounts → Generate new private key
 - Download JSON → copy `project_id`, `client_email`, `private_key`
3. Firebase Console → Authentication → Users → "Add user"
 - Email: your email, password: strong password
 - After creation: copy the **UID** from the Users table — this is `ADMIN_UID`

✓ You have all values needed for `.env.local`.

TASK 0.4 — Environment Variables

Time: 10 min

Create `.env.local` at project root. Create `.env.example` with same keys but empty values.

```
bash
```

```
# .env.local
NEXT_PUBLIC_FIREBASE_API_KEY=
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=
NEXT_PUBLIC_FIREBASE_PROJECT_ID=
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=
NEXT_PUBLIC_FIREBASE_APP_ID=

FIREBASE_ADMIN_PROJECT_ID=
FIREBASE_ADMIN_CLIENT_EMAIL=
FIREBASE_ADMIN_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\n...\n-----END PRIVATE KEY-----"

ADMIN_UID=

GITHUB_TOKEN=

EMAILJS_SERVICE_ID=
EMAILJS_TEMPLATE_ID=
EMAILJS_PUBLIC_KEY=
CONTACT_EMAIL=
```

Add `.env.local` to `.gitignore` (should already be there from create-next-app).

☒ `process.env.NEXT_PUBLIC_FIREBASE_PROJECT_ID` is accessible in the app.

TASK 0.5 — Git Hygiene & Tooling

Time: 40 min

Configure Prettier:

```
json
// .prettierrc
{
  "semi": true,
  "singleQuote": true,
  "tabWidth": 2,
  "trailingComma": "es5",
  "printWidth": 100
}
```

Configure lint-staged in `package.json`:

```
json

"lint-staged": {
  "*.ts,tsx": ["eslint --fix", "prettier --write"],
  "*.json,md,css": ["prettier --write"]
}
```

Setup Husky:

```
bash

npx husky init
echo "npx lint-staged && npm run type-check" > .husky/pre-commit
chmod +x .husky/pre-commit
```

Add scripts to `package.json`:

```
json

"scripts": {
  "dev": "next dev",
  "build": "next build",
  "start": "next start",
  "lint": "next lint",
  "type-check": "tsc --noEmit",
  "format": "prettier --write .",
  "format:check": "prettier --check ."
}
```

Initial commit:

```
bash

git add .
git commit -m "chore: project scaffold, dependencies, tooling"
git push origin main
```

✅ Pre-commit hook runs on `git commit`. `npm run type-check` passes with zero errors.

TASK 0.6 — Branching Strategy

Time: 10 min

```
bash

git checkout -b dev
git push origin dev
```

Go to GitHub → Settings → Branches → Add rule:

- Branch name pattern: `main`
- Check: "Require a pull request before merging"
- Check: "Require status checks to pass" (add later once CI is configured)

From this point forward: all work is done on `feature/*` branches, merged into `dev`, then `dev` → `main` for production.

```
bash

git checkout -b feature/foundation
```

✓ Branching strategy in place. Never commit directly to `main`.

PHASE 1A: Foundation Layer (~14 hours)

TASK 1.1 — TypeScript Types (All Interfaces)

Time: 45 min

File: `types/index.ts`

Define every interface from the TRD Section 13 in one file. This is the contract for the entire app. Do this **FIRST** — before any Firebase code — so TypeScript catches mismatches everywhere.

```
typescript
```

```
// types/index.ts

import { Timestamp } from 'firebase/firestore';

export type TechCategory = 'language' | 'framework' | 'tool' | 'cloud' | 'database'

export interface TechStackItem {
  tag: string;
  category: TechCategory;
}

export type DevlogEntryType =
  | 'milestone'
  | 'daily_log'
  | 'issue'
  | 'fix'
  | 'architecture'
  | 'reflection';

export interface Project {
  id: string;
  title: string;
  slug: string;
  shortDescription: string;
  fullDescription: string;
  coverImageUrl: string | null;
  techStack: TechStackItem[];
  githubRepo: string | null;
  loomVideoIdDemo: string | null;
  loomVideoIdArch: string | null;
  linkedinUrl: string | null;
  liveDemoUrl: string | null;
  projectStartDate: Timestamp | null;
  projectEndDate: Timestamp | null;
  isOngoing: boolean;
  projectStatus: 'active' | 'completed' | 'archived';
  tier: 0 | 1 | 2 | 3;
  order: number;
  featured: boolean;
  published: boolean;
  createdAt: Timestamp;
  updatedAt: Timestamp;
}
```

```
export interface DevlogEntry {
  id: string;
  type: DevlogEntryType;
  title: string;
  content: string;
  entryDate: Timestamp;
  linkedEntryId: string | null;
  published: boolean;
  createdAt: Timestamp;
  updatedAt: Timestamp;
}

export interface DevlogEntryVersion {
  id: string;
  content: string;
  savedAt: Timestamp;
}

export interface ProjectComment {
  id: string;
  authorName: string;
  content: string;
  status: 'pending' | 'approved' | 'rejected';
  ipHash: string;
  createdAt: Timestamp;
}

export interface TimelineEvent {
  id: string;
  type: 'academic' | 'corporate' | 'achievement';
  title: string;
  organization: string;
  location: string;
  dateRange: string;
  startDate: Timestamp;
  endDate: Timestamp | null;
  description: string[];
  logoUrl: string | null;
  order: number;
}

export interface ContactSubmission {
  id: string;
```



```
name: string;
email: string;
company: string | null;
message: string;
status: 'unread' | 'read' | 'archived';
emailDelivered: boolean;
ipHash: string;
userAgent: string;
createdAt: Timestamp;
}

export interface GlobalSettings {
  displayName: string;
  tagline: string;
  subHeadline: string;
  avatarUrl: string;
  cvFileUrl: string;
  email: string;
  github: string;
  linkedin: string;
  twitter: string | null;
  seoTitleSuffix: string;
  metaDescription: string;
  updatedAt: Timestamp;
}

export type AuditEvent =
  | 'login_success' | 'login_failure' | 'logout'
  | 'project_created' | 'project_updated' | 'project_deleted'
  | 'project_tier_changed'
  | 'devlog_entry_created' | 'devlog_entry_updated' | 'devlog_entry_deleted'
  | 'comment_approved' | 'comment_deleted'
  | 'cv_uploaded' | 'settings_updated';

export interface AuditLogEntry {
  id: string;
  event: AuditEvent;
  adminUid: string | null;
  ipHash: string;
  userAgent: string;
  metadata: Record<string, unknown>;
  createdAt: Timestamp;
}
```

✓ npm run type-check passes. Zero any types.

TASK 1.2 — Firebase Client SDK Init

Time: 20 min

File: lib/firebase/client.ts

typescript

```
// lib/firebase/client.ts
import { initializeApp, getApps, getApp } from 'firebase/app';
import { getFirestore } from 'firebase/firestore';
import { getAuth } from 'firebase/auth';
import { getStorage } from 'firebase/storage';

const firebaseConfig = {
  apiKey: process.env.NEXT_PUBLIC_FIREBASE_API_KEY,
  authDomain: process.env.NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN,
  projectId: process.env.NEXT_PUBLIC_FIREBASE_PROJECT_ID,
  storageBucket: process.env.NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: process.env.NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID,
  appId: process.env.NEXT_PUBLIC_FIREBASE_APP_ID,
};

const app = getApps().length > 0 ? getApp() : initializeApp(firebaseConfig);

export const db = getFirestore(app);
export const auth = getAuth(app);
export const storage = getStorage(app);
```

✓ No console errors when importing this file.

TASK 1.3 — Firebase Admin SDK Init

Time: 20 min

File: lib/firebase/admin.ts

typescript

```
// lib/firebase/admin.ts
// THIS FILE MUST NEVER BE IMPORTED IN CLIENT COMPONENTS
import { initializeApp, getApps, cert, App } from 'firebase-admin/app';
import { getFirestore } from 'firebase-admin/firestore';
import { getAuth } from 'firebase-admin/auth';

function createAdminApp(): App {
  if (getApps().length > 0) return getApps()[0];

  return initializeApp({
    credential: cert({
      projectId: process.env.FIREBASE_ADMIN_PROJECT_ID,
      clientEmail: process.env.FIREBASE_ADMIN_CLIENT_EMAIL,
      // Replace literal \n from env var string with actual newlines
      privateKey: process.env.FIREBASE_ADMIN_PRIVATE_KEY?.replace(/\\n/g, '\n'),
    }),
  });
}

const adminApp = createAdminApp();

export const adminDb = getFirestore(adminApp);
export const adminAuth = getAuth(adminApp);
```

✓ Admin SDK initializes without error in a Server Action test.

TASK 1.4 — Firestore Type-Safe Converters

Time: 30 min

File: `lib/firebase/converters.ts`

typescript

```
// lib/firebase/converters.ts
import {
  FirestoreDataConverter,
  QueryDocumentSnapshot,
  SnapshotOptions,
  WithFieldValue,
  DocumentData,
} from 'firebase/firestore';
import { Project, DevlogEntry, DevlogEntryVersion, ProjectComment, TimelineEvent, GlobalSettings } from '../models';

function makeConverter<T extends { id: string }>(): FirestoreDataConverter<T> {
  return {
    toFirestore(data: WithFieldValue<T>): DocumentData {
      const { id, ...rest } = data as T;
      return rest;
    },
    fromFirestore(snapshot: QueryDocumentSnapshot, options: SnapshotOptions): T {
      return { id: snapshot.id, ...snapshot.data(options) } as T;
    },
  };
}

export const projectConverter = makeConverter<Project>();
export const devlogEntryConverter = makeConverter<DevlogEntry>();
export const devlogVersionConverter = makeConverter<DevlogEntryVersion>();
export const commentConverter = makeConverter<ProjectComment>();
export const timelineConverter = makeConverter<TimelineEvent>();
export const settingsConverter = makeConverter<GlobalSettings>();
```

✅ Converters compile. TypeScript infers correct types from Firestore queries using these converters.

TASK 1.5 — Firestore Security Rules

Time: 30 min

File: `firestore.rules` (project root)

Copy the exact rules from TRD Section 14. Replace `YOUR_ADMIN_UID` with your actual UID — BUT only in the deployed version. In the file committed to Git, keep it as the placeholder to avoid leaking your UID publicly.

Deploy rules:

```
bash
```

```
# Install Firebase CLI if not already installed
npm install -g firebase-tools
firebase login
firebase init firestore # Select existing project, use existing firestore.rules
firebase deploy --only firestore:rules
```

Then deploy Storage rules:

```
bash
```

```
firebase init storage # Use existing storage.rules
firebase deploy --only storage
```

✅ Test in Firebase Console Firestore → Rules Playground: a read to `projects/test` with no auth returns "denied." A read with the admin UID returns "allowed."

TASK 1.6 — Firestore Indexes

Time: 20 min

File: `firestore.indexes.json`

```
json
```

```

{
  "indexes": [
    {
      "collectionGroup": "projects",
      "queryScope": "COLLECTION",
      "fields": [
        { "fieldPath": "published", "order": "ASCENDING" },
        { "fieldPath": "tier", "order": "ASCENDING" },
        { "fieldPath": "order", "order": "ASCENDING" }
      ]
    },
    {
      "collectionGroup": "devlog",
      "queryScope": "COLLECTION",
      "fields": [
        { "fieldPath": "published", "order": "ASCENDING" },
        { "fieldPath": "entryDate", "order": "ASCENDING" }
      ]
    },
    {
      "collectionGroup": "comments",
      "queryScope": "COLLECTION",
      "fields": [
        { "fieldPath": "status", "order": "ASCENDING" },
        { "fieldPath": "createdAt", "order": "DESCENDING" }
      ]
    }
  ],
  "fieldOverrides": []
}

```

```
bash
```

```
firebase deploy --only firestore:indexes
```

✅ Indexes show as "Building" then "Enabled" in Firebase Console.

TASK 1.7 — Seed Initial Firestore Data

Time: 30 min

In Firebase Console → Firestore → Start collection:

1. **Collection:** `settings` → Document ID: `global`

- Add all fields from `GlobalSettings` interface with your real data
- `displayName`: "Your Name"
- `tagline`: "Your tagline"
- `linkedin`: "https://linkedin.com/in/yourhandle"
- `github`: "https://github.com/yourusername"
- etc.

2. **Collection:** `projects` → Add one test document:

- `title`: "Test Project"
- `slug`: "test-project"
- `published`: true
- `tier`: 1
- `order`: 1
- All other fields: their null/false/empty defaults

✅ Test project document readable from Firebase Console.

TASK 1.8 — Tailwind Config & Design Tokens

Time: 40 min

File: `tailwind.config.ts`

typescript

```
import type { Config } from 'tailwindcss';

const config: Config = {
  content: [
    './pages/**/*..{js,ts,jsx,tsx,mdx}',
    './components/**/*..{js,ts,jsx,tsx,mdx}',
    './app/**/*..{js,ts,jsx,tsx,mdx}',
  ],
  darkMode: 'class',
  theme: {
    extend: {
      colors: {
        // Define your brand colors here – used throughout
        brand: {
          50: '#f0f9ff',
          500: '#0ea5e9',
          600: '#0284c7',
          900: '#0c4a6e',
        },
        // Devlog entry type colors
        devlog: {
          milestone: '#f59e0b',
          daily: '#6366f1',
          issue: '#ef4444',
          fix: '#10b981',
          architecture: '#3b82f6',
          reflection: '#8b5cf6',
        },
      },
      fontFamily: {
        sans: ['var(--font-geist-sans)'],
        mono: ['var(--font-geist-mono)'],
      },
      animation: {
        'fade-in': 'fadeIn 0.5s ease-in-out',
        'slide-up': 'slideUp 0.4s ease-out',
        'spin-slow': 'spin 3s linear infinite',
      },
      keyframes: {
        fadeIn: {
          '0%': { opacity: '0' },
          '100%': { opacity: '1' },
        },
      },
    },
  },
};
```



```
      slideUp: {
        '0%': { opacity: '0', transform: 'translateY(20px)' },
        '100%': { opacity: '1', transform: 'translateY(0)' },
      },
    },
  },
},
plugins: [],
};

export default config;
```

✅ Custom colors accessible via `bg-brand-500`, `text-devlog-fix`, etc.

TASK 1.9 — Root Layout & Dark Mode Setup

Time: 45 min

File: `app/layout.tsx`

typescript

```
// app/layout.tsx
import type { Metadata } from 'next';
import { GeistSans } from 'geist/font/sans';
import { GeistMono } from 'geist/font/mono';
import { Analytics } from '@vercel/analytics/react';
import './globals.css';

// Dark mode init script – runs before React hydration to prevent FOUC
const themeScript = `
(function() {
  const stored = localStorage.getItem('theme');
  const prefersDark = window.matchMedia('(prefers-color-scheme: dark)').matches;
  if (stored === 'dark' || (!stored && prefersDark)) {
    document.documentElement.classList.add('dark');
  }
})();
`;

export const metadata: Metadata = {
  title: { template: '%s | Your Name', default: 'Your Name – AI Engineer' },
  description: 'AI Engineer building production-grade LLM systems.',
};

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en" suppressHydrationWarning>
      <head>
        <script dangerouslySetInnerHTML={{ __html: themeScript }} />
      </head>
      <body className={`${GeistSans.variable} ${GeistMono.variable} font-sans bg-white}
        {children}
        <Analytics />
      </body>
    </html>
  );
}
```

Install geist font: `npm install geist`

Create `lib/hooks/useTheme.ts`:

```
typescript
```

```
'use client';
import { useEffect, useState } from 'react';

export function useTheme() {
  const [theme, setTheme] = useState<'light' | 'dark'>('light');

  useEffect(() => {
    const stored = localStorage.getItem('theme') as 'light' | 'dark' | null;
    setTheme(stored ?? (window.matchMedia('(prefers-color-scheme: dark)').matches ?
    }, []));

    const toggle = () => {
      const next = theme === 'light' ? 'dark' : 'light';
      setTheme(next);
      localStorage.setItem('theme', next);
      document.documentElement.classList.toggle('dark', next === 'dark');
    };

    return { theme, toggle };
  }
}
```

✓ Dark mode toggle works. No flash of wrong theme on page load.

TASK 1.10 — Next.js Config (Performance & Security)

Time: 20 min

File: `next.config.ts`

typescript

```

import type { NextConfig } from 'next';

const nextConfig: NextConfig = {
  images: {
    remotePatterns: [
      { protocol: 'https', hostname: 'firebasestorage.googleapis.com' },
      { protocol: 'https', hostname: 'avatars.githubusercontent.com' },
    ],
  },
  experimental: {
    optimizePackageImports: ['lucide-react', 'framer-motion'],
  },
  async headers() {
    return [
      {
        source: '/(.*)',
        headers: [
          { key: 'X-Frame-Options', value: 'SAMEORIGIN' },
          { key: 'X-Content-Type-Options', value: 'nosniff' },
          { key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' },
          {
            key: 'Strict-Transport-Security',
            value: 'max-age=63072000; includeSubDomains; preload',
          },
        ],
      },
      {
        source: '/admin/:path*',
        headers: [{ key: 'X-Robots-Tag', value: 'noindex, nofollow' }],
      },
    ];
  },
};

export default nextConfig;

```

✓ `npm run build` completes without errors.

PHASE 1B: Admin Auth System (~8 hours)

This is the backbone of the entire admin. Build it completely before ANY admin UI.

TASK 2.1 — Session Verification Utility

Time: 30 min

File: `lib/auth/verify-session.ts`

Copy the exact implementation from TRD Section 15.3 verbatim. This function will be called at the top of every Server Action and every admin Server Component.

✅ Function exists and compiles. TypeScript return type is `DecodedIdToken`.

TASK 2.2 — Auth Route Handler (Set Cookie)

Time: 30 min

File: `app/api/auth/session/route.ts`

typescript

```

import { NextRequest, NextResponse } from 'next/server';
import { adminAuth } from '@lib/firebase/admin';
import { cookies } from 'next/headers';

export async function POST(req: NextRequest) {
  const { token } = await req.json();

  if (!token) return NextResponse.json({ error: 'No token' }, { status: 400 });

  try {
    const decoded = await adminAuth.verifyIdToken(token);

    if (decoded.uid !== process.env.ADMIN_UID) {
      return NextResponse.json({ error: 'Access denied' }, { status: 403 });
    }

    if (!decoded.email_verified) {
      return NextResponse.json({ error: 'Email not verified' }, { status: 403 });
    }

    const cookieStore = await cookies();
    cookieStore.set('__session', token, {
      httpOnly: true,
      secure: process.env.NODE_ENV === 'production',
      sameSite: 'strict',
      maxAge: 3600,
      path: '/admin',
    });

    return NextResponse.json({ success: true });
  } catch {
    return NextResponse.json({ error: 'Invalid token' }, { status: 401 });
  }
}

```

✓ POST to `/api/auth/session` with a valid token sets the cookie.

TASK 2.3 — Logout Route Handler

Time: 15 min

File: `app/api/auth/logout/route.ts`

typescript

```
import { NextResponse } from 'next/server';
import { cookies } from 'next/headers';

export async function POST() {
  const cookieStore = await cookies();
  cookieStore.delete('__session');
  return NextResponse.redirect(new URL('/admin/login', process.env.NEXTAUTH_URL ?? ''));
}
```

✓ POST to `/api/auth/logout` deletes cookie and redirects.

TASK 2.4 — Middleware (Route Guard)

Time: 30 min

File: `middleware.ts` (project root)

typescript

```
import { NextRequest, NextResponse } from 'next/server';

export function middleware(req: NextRequest) {
  const { pathname } = req.nextUrl;

  // Only guard /admin routes (except /admin/login)
  if (pathname.startsWith('/admin') && !pathname.startsWith('/admin/login')) {
    const session = req.cookies.get('__session');
    if (!session) {
      return NextResponse.redirect(new URL('/admin/login', req.url));
    }
  }

  return NextResponse.next();
}

export const config = {
  matcher: ['/admin/:path*'],
};
```

✓ Visiting `/admin/dashboard` with no cookie redirects to `/admin/login`.

TASK 2.5 — Audit Logger Helper

Time: 20 min

File: `lib/audit.ts`

typescript

```
// lib/audit.ts - server-only
import { adminDb } from '@lib/firebase/admin';
import { FieldValue } from 'firebase-admin/firestore';
import type { AuditEvent } from '@types';

interface AuditParams {
  event: AuditEvent;
  adminUid?: string;
  ipHash?: string;
  userAgent?: string;
  metadata?: Record<string, unknown>;
}

export async function logAuditEvent({
  event,
  adminUid = null,
  ipHash = '',
  userAgent = '',
  metadata = {},
}: AuditParams) {
  try {
    await adminDb.collection('admin_audit_log').add({
      event,
      adminUid,
      ipHash,
      userAgent,
      metadata,
      createdAt: FieldValue.serverTimestamp(),
    });
  } catch (err) {
    // Audit logging failures should never crash the main operation
    console.error('[audit] Failed to log event:', err);
  }
}
```

TASK 2.6 — Rate Limiter

Time: 30 min

File: `lib/rate-limit.ts`

typescript

```

// lib/rate-limit.ts - server-only
import { adminDb } from '@lib/firebase/admin';
import { FieldValue, Timestamp } from 'firebase-admin/firestore';
import crypto from 'crypto';

interface RateLimitResult {
  allowed: boolean;
  blocked: boolean;
  blockedUntil?: Date;
}

export function hashIp(ip: string): string {
  return crypto.createHash('sha256').update(ip + process.env.ADMIN_UID).digest('hex')
}

export async function checkRateLimit(
  ip: string,
  endpoint: string,
  maxRequests: number,
  windowSeconds: number,
  lockoutSeconds: number
): Promise<RateLimitResult> {
  const id = hashIp(ip + endpoint);
  const ref = adminDb.collection('rate_limits').doc(id);

  return adminDb.runTransaction(async (tx) => {
    const doc = await tx.get(ref);
    const now = Timestamp.now();

    if (!doc.exists) {
      tx.set(ref, { count: 1, windowStart: now, blockedUntil: null });
      return { allowed: true, blocked: false };
    }

    const data = doc.data()!;

    // Check if currently blocked
    if (data.blockedUntil && data.blockedUntil.toDate() > now.toDate()) {
      return { allowed: false, blocked: true, blockedUntil: data.blockedUntil.toDate() };
    }

    // Check if window has expired → reset
    const windowAge = now.toDate().getTime() - data.windowStart.toDate().getTime();
  });
}

```

```
if (windowAge > windowSeconds * 1000) {
  tx.set(ref, { count: 1, windowStart: now, blockedUntil: null });
  return { allowed: true, blocked: false };
}

// Increment count
const newCount = data.count + 1;

if (newCount > maxRequests) {
  const blockedUntil = new Date(now.toDate().getTime() + lockoutSeconds * 1000);
  tx.update(ref, { count: newCount, blockedUntil: Timestamp.fromDate(blockedUntil) });
  return { allowed: false, blocked: true, blockedUntil };
}

tx.update(ref, { count: newCount });
return { allowed: true, blocked: false };
});
}
```

✓ Calling `checkRateLimit` 6 times returns `blocked: true` on the 6th call.

TASK 2.7 — Admin Login Page (Full Flow)

Time: 90 min

Files: `app/admin/login/page.tsx`, `lib/actions/auth.ts`

Server Action:

typescript

```
// lib/actions/auth.ts
'use server';

export async function setAdminSession(token: string) {
  const { adminAuth } = await import('@lib/firebase/admin');
  const { cookies } = await import('next/headers');
  const decoded = await adminAuth.verifyIdToken(token);

  if (decoded.uid !== process.env.ADMIN_UID || !decoded.email_verified) {
    throw new Error('Access denied');
  }

  const cookieStore = await cookies();
  cookieStore.set('__session', token, {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'strict',
    maxAge: 3600,
    path: '/admin',
  });
}
```

Login Page (`app/admin/login/page.tsx`):

- Client component
- Form: email + password fields
- On submit: `signInWithEmailAndPassword` → `getIdToken()` → call `setAdminSession`
Server Action → redirect to `/admin/dashboard`
- Error states: "Invalid credentials" (generic), "Too many attempts"
- No UI chrome (no sidebar) — isolated login page

✅ Entering correct credentials → redirected to `/admin/dashboard`. Entering wrong credentials → error message shown. After 5 bad attempts → rate limit message with unlock time.

TASK 2.8 — Admin Layout (Shell)

Time: 45 min

Files: `app/admin/layout.tsx`, `components/admin/AdminSidebar.tsx`

`app/admin/layout.tsx` is a Server Component that:

`AdminSidebar` is a Client Component with navigation links to all admin sections. Uses `usePathname()` to highlight the active link.

Sidebar items (with icons from lucide-react):

- Dashboard (LayoutDashboard)
- Projects (FolderCode)
- Comments (MessageSquare) + pending badge
- Timeline (Calendar)
- Inbox (Inbox) + unread badge
- CV Upload (FileUp)
- Settings (Settings)
- Logout button (Logout)

✅ Admin layout renders on all `/admin/*` routes. Navigating between sections works. Logout clears cookie and redirects to login.

PHASE 1C: Data Layer — Server Actions (~10 hours)

Build all Server Actions before building any public pages. Pages will just call these.

TASK 3.1 — Project Server Actions

Time: 90 min

File: `lib/actions/projects.ts`

Implement in this exact order:

1. `getPublishedProjects(tier?: 0|1|2|3)` — reads `projects` collection filtered by `published: true` + optional tier, ordered by `order ASC`. Used for homepage and `/projects` index.
2. `getProjectBySlug(slug: string)` — reads single project document by slug. Returns `null` if not found or not published. Used for static generation.
3. `getAllProjectSlugs()` — returns array of `{ slug: string }` for all published projects. Used by `generateStaticParams`.

4. `getAllProjectsAdmin()` — returns ALL projects (including drafts, all tiers) for admin panel.
 5. `upsertProject(data, projectId?)` — full implementation from TRD 16.2. Key points:
 - `verifyAdminSession()` first
 - Zod validation (define `ProjectFormSchema` in this file)
 - Auto-generate slug from title using `slugify()` utility if not provided
 - Check slug uniqueness (query Firestore for existing slug, reject if taken by different project)
 - On tier change: log `project_tier_changed` audit event with `{ fromTier, toTier, projectId }`
 - Call `revalidatePath` for homepage, `/projects`, and specific project slug
 6. `changeProjectTier(projectId, newTier)` — lightweight action for the inline tier dropdown in admin list. Optimized for speed — only updates `tier` and `updatedAt`.
 7. `deleteProject(projectId)` — `verifyAdminSession`, delete main document (does NOT cascade-delete sub-collections — that's a separate cleanup), `revalidatePath`.
- ✅ All functions compile. `getPublishedProjects(1)` returns the test project seeded in Task 1.7. `upsertProject` creates a new project document in Firestore.
-

TASK 3.2 — Devlog Server Actions (with Auto-Versioning)

Time: 60 min

File: `lib/actions/devlog.ts`

1. `getDevlogEntries(projectId, adminView?)` — public: `published: true`, ordered by `entryDate ASC`. Admin: all entries, `createdAt DESC`.
2. `upsertDevlogEntry(projectId, data, entryId?)` — the critical action:

```

- verifyAdminSession()
- Zod validate
- If entryId provided (edit):
  a. Fetch current entry from Firestore (adminDb, bypasses rules)
  b. Check: if new content === current content → skip version write (no
duplicate)
  c. Count versions in sub-collection
  d. If count >= 50 → query oldest version doc → delete it (batch with next
write)
  e. Write snapshot to versions/{auto-id}: { content: currentContent, savedAt:
serverTimestamp() }
  f. Update entry with new data + updatedAt: serverTimestamp()
- If no entryId (create):
  a. Write new entry document
- Log audit event
- revalidatePath(/projects/${slug})

```

3. `deleteDevlogEntry(projectId, entryId)` — admin only, no cascade needed (versions sub-collection can be orphaned — Firebase handles this)
4. `getDevlogVersions(projectId, entryId)` — admin only, returns all versions sorted by `savedAt DESC`

✅ Creating an entry, editing it twice, then calling `getDevlogVersions` returns 2 version documents (the original and first edit).

TASK 3.3 — Comment Server Actions

Time: 30 min

File: `lib/actions/comments.ts`

1. `submitComment(projectId, data)` — public action, rate-limited (3 per IP per hour), creates comment with `status: 'pending'`, validates with Zod (20-1000 chars).
2. `getApprovedComments(projectId)` — public, returns `status: 'approved'` only.
3. `getAllPendingComments()` — admin only, queries across all projects. Returns comments with their `projectId` for the moderation queue.
4. `approveComment(projectId, commentId)` — admin only, updates status to `'approved'`, revalidates project page.
5. `deleteComment(projectId, commentId)` — admin only.

✓ Submitting a comment creates a `pending` doc. Approving it updates status to `approved`.
Public query only returns `approved` comments.

TASK 3.4 — Timeline Server Actions

Time: 30 min

File: `lib/actions/timeline.ts`

1. `getTimelineEvents()` — public, returns all events ordered by `startDate DESC`
2. `upsertTimelineEvent(data, eventId?)` — admin only
3. `deleteTimelineEvent(eventId)` — admin only

✓ Create a test timeline event from admin. It appears in `getTimelineEvents()`.

TASK 3.5 — Contact Form Server Action

Time: 30 min

File: `lib/actions/contact.ts`

typescript


```

'use server';

import { z } from 'zod';
import { adminDb } from '@lib/firebase/admin';
import { checkRateLimit, hashIp } from '@lib/rate-limit';
import { FieldValue } from 'firebase-admin/firestore';
import { headers } from 'next/headers';

const ContactSchema = z.object({
  name: z.string().min(1).max(100),
  email: z.string().email(),
  company: z.string().optional(),
  message: z.string().min(20).max(2000),
});

export async function submitContactForm(formData: z.infer<typeof ContactSchema>) {
  const headersList = await headers();
  const ip = headersList.get('x-forwarded-for') ?? 'unknown';

  // Rate limit: 3 submissions per IP per hour
  const rateCheck = await checkRateLimit(ip, 'contact', 3, 3600, 3600);
  if (!rateCheck.allowed) {
    return { success: false, error: 'rate_limited' as const };
  }

  const parsed = ContactSchema.safeParse(formData);
  if (!parsed.success) {
    return { success: false, error: 'validation_failed' as const };
  }

  await adminDb.collection('contact_submissions').add({
    ...parsed.data,
    status: 'unread',
    emailDelivered: false, // Updated after EmailJS sends
    ipHash: hashIp(ip),
    userAgent: headersList.get('user-agent') ?? '',
    createdAt: FieldValue.serverTimestamp(),
  });

  return { success: true };
}

```

✓ Calling `submitContactForm` 4 times from the same IP returns `rate_limited` on the 4th.

TASK 3.6 — Settings Server Actions

Time: 20 min

File: `lib/actions/settings.ts`

1. `getGlobalSettings()` — public, reads `settings/global` document
2. `updateGlobalSettings(data)` — admin only, updates the document, revalidates homepage

✅ Both actions work. `getGlobalSettings` returns the data seeded in Task 1.7.

TASK 3.7 — GitHub Stats Route Handler

Time: 30 min

File: `app/api/github-stats/route.ts`

typescript

```

import { NextRequest, NextResponse } from 'next/server';

export const revalidate = 3600;

export async function GET(req: NextRequest) {
  const repo = req.nextUrl.searchParams.get('repo');

  if (!repo || !repo.includes('/')) {
    return NextResponse.json({ stars: null, commits: null, error: 'Invalid repo form' });
  }

  try {
    const headers = {
      Authorization: `Bearer ${process.env.GITHUB_TOKEN}`,
      Accept: 'application/vnd.github.v3+json',
    };

    const [repoRes, commitsRes] = await Promise.allSettled([
      fetch(`https://api.github.com/repos/${repo}`, { headers }),
      fetch(`https://api.github.com/repos/${repo}/commits?per_page=1`, { headers }),
    ]);

    const stars = repoRes.status === 'fulfilled' && repoRes.value.ok
      ? (await repoRes.value.json()).stargazers_count
      : null;

    // GitHub returns total commit count in Link header
    let commits: number | null = null;
    if (commitsRes.status === 'fulfilled' && commitsRes.value.ok) {
      const linkHeader = commitsRes.value.headers.get('Link');
      const match = linkHeader?.match(/page=(\d+)>; rel="last"/);
      commits = match ? parseInt(match[1]) : null;
    }

    return NextResponse.json({ stars, commits });
  } catch {
    return NextResponse.json({ stars: null, commits: null, error: true });
  }
}

```

✓ GET /api/github-stats?repo=username/repo returns { stars: N, commits: N }.

PHASE 1D: Public Pages (~18 hours)

TASK 4.1 — Shared UI Primitives

Time: 60 min

Directory: `components/ui/`

Build these small, reusable components. Keep them simple — they just enforce consistency:

- `Button.tsx` — variants: `primary`, `secondary`, `ghost`, `danger`. Sizes: `sm`, `md`, `lg`
- `Badge.tsx` — for tech stack tags. Color varies by `TechCategory`
- `Modal.tsx` — accessible dialog with backdrop click to close, escape key support
- `Toast.tsx` — success/error/info toasts, auto-dismiss after 4 seconds
- `Skeleton.tsx` — loading placeholder shapes
- `Tabs.tsx` — tabbed interface with keyboard navigation
- `MarkdownRenderer.tsx` — wraps `react-markdown` + `remark-gfm` + `rehype-highlight` with consistent prose styling

✓ All primitives render without errors in a test page.

TASK 4.2 — Navbar Component

Time: 30 min

File: `components/public/Navbar.tsx`

- Sticky top, blurs on scroll
- Logo/name on left (links to `#home`)
- Nav links: Home, Projects, Contact
- Dark mode toggle (sun/moon icon)
- Mobile hamburger menu with animated drawer
- CV download button (links to `settings.cvFileUrl`)

✓ Navbar renders. Dark mode toggle works. Mobile menu opens/closes smoothly.

TASK 4.3 — Hero Section

Time: 90 min

File: `components/public/Hero/HeroSection.tsx`

Layout:

```
| Hello, I'm [Name]
| [Tagline - animated typewriter effect]
| [Sub-headline]
|
| [Download CV] [View Projects] [Contact]
|
| [Skills Matrix - SVG grid of tech logos/names]
```

Data comes from `getGlobalSettings()` called in the parent Server Component `(app/page.tsx)`.

Skills Matrix (`components/public/Hero/SkillsMatrix.tsx`):

- SVG-based grid
- Each cell: icon + label for a technology
- Subtle hover animation (scale + border glow)
- Hardcode the tech list here (it's your skills, not CMS-managed)
- Animate cells in with staggered `fadeIn` on mount

✅ Hero section renders with your name and tagline. Skills matrix animates on load.

TASK 4.4 — Experience Timeline

Time: 90 min

File: `components/public/Timeline/ExperienceTimeline.tsx`

Data: fetched via `getTimelineEvents()` in `(app/page.tsx)`.

- Vertical timeline with alternating left/right layout on desktop, single column on mobile
- Each event card: logo, title, organization, date range, bullet descriptions
- Scroll-triggered reveal animations using Framer Motion `whileInView`
- Type icons: graduation cap for academic, building for corporate, trophy for achievement

✅ Timeline renders all seeded events. Scroll reveal works.

TASK 4.5 — Homepage Project Sections (Tier 1/2/3)

Time: 3 hours

Files:

- `components/public/Projects/Tier1Card.tsx`
- `components/public/Projects/Tier2Card.tsx`
- `components/public/Projects/Tier3Row.tsx`
- `components/public/Projects/ProjectsSection.tsx`

Tier 1 Card:

```
| [Cover Image / Loom Embed - 16:9] |  
| [Title] [Tech stack badges] |  
| [Description - 3 sentences] |  
| ★ N stars 📄 N commits |  
| [View Project] [GitHub] [LinkedIn] [Demo?] |
```

GitHub stats: fetched server-side in `app/page.tsx` via internal fetch to `/api/github-stats`.
Pass as prop to the card. On `null`: show `[-]`.

Tier 2 Card: Simpler. Image + title + 1-line description + tags + "View Project" button. On hover: overlay shows GitHub + LinkedIn icon links.

Tier 3 Row: Single line. Name | description | GitHub icon | LinkedIn icon.

ProjectsSection composes all three, with section headers and dividers.

✅ All three tier sections render correctly with the test project. Tier 1 shows GitHub stats.

TASK 4.6 — Contact Section

Time: 60 min

File: `components/public/ContactForm.tsx`

- Client component using React Hook Form + Zod
- Fields: Name, Email, Company (optional), Message
- On submit: calls `submitContactForm` Server Action
- Loading state during submission (disable form + spinner)
- Success: "Message sent! I'll be in touch soon." toast
- Error states: validation errors inline, rate limit message, generic failure

✅ Form submits successfully. Contact submission appears in Firestore. Rate limit works after 3 submissions.

TASK 4.7 — Homepage Assembly (`app/page.tsx`)

Time: 45 min

typescript

```

// app/page.tsx
import { Suspense } from 'react';
import { getGlobalSettings } from '@lib/actions/settings';
import { getPublishedProjects } from '@lib/actions/projects';
import { getTimelineEvents } from '@lib/actions/timeline';

export const revalidate = 300; // 5 minutes ISR

export default async function HomePage() {
  const [settings, tier1, tier2, tier3, timeline] = await Promise.all([
    getGlobalSettings(),
    getPublishedProjects(1),
    getPublishedProjects(2),
    getPublishedProjects(3),
    getTimelineEvents(),
  ]);

  // Fetch GitHub stats for tier 1 projects server-side
  const githubStats = await Promise.allSettled(
    tier1.map(p =>
      p.githubRepo
        ? fetch(`${process.env.NEXT_PUBLIC_BASE_URL}/api/github-stats?repo=${p.githubRepo}`)
          .then(r => r.json())
          : Promise.resolve({ stars: null, commits: null })
    )
  );

  return (
    <main>
      <Navbar settings={settings} />
      <HeroSection settings={settings} />
      <ExperienceTimeline events={timeline} />
      <ProjectsSection tier1={tier1} tier2={tier2} tier3={tier3} githubStats={githubStats} />
      <ContactForm />
      <Footer settings={settings} />
    </main>
  );
}

```

Add to `next.config.ts`: `env: { NEXT_PUBLIC_BASE_URL: process.env.VERCEL_URL ? ... : 'http://localhost:3000' }`

✓ Homepage renders with all sections. ISR tag is set to 300 seconds.

TASK 4.8 — /projects Index Page

Time: 60 min

File: `app/projects/page.tsx`

- Fetches all published projects (all tiers)
- Client-side filtering by tech tag and tier using `useState`
- Paginated grid (12 per page) with prev/next controls
- Each card uses `Tier2Card` component (unified display)
- SSG shell with ISR 300s

✓ `/projects` shows all published projects. Filter by tech tag works.

TASK 4.9 — Individual Project Page Shell

Time: 2 hours

Files:

- `app/projects/[slug]/page.tsx`
- `components/public/ProjectPage/ProjectHeader.tsx`

typescript

```
// app/projects/[slug]/page.tsx
export async function generateStaticParams() {
  const slugs = await getAllProjectSlugs();
  return slugs.map(({ slug }) => ({ slug }));
}

export async function generateMetadata({ params }) {
  const project = await getProjectBySlug(params.slug);
  if (!project) return { title: 'Not Found' };
  return {
    title: project.title,
    description: project.shortDescription,
    openGraph: {
      title: project.title,
      description: project.shortDescription,
      images: [project.coverImageUrl ?? '/og-default.png'],
      type: 'article',
    },
  };
}

export default async function ProjectPage({ params }) {
  const project = await getProjectBySlug(params.slug);
  if (!project) notFound();

  const [devlogEntries, comments] = await Promise.all([
    getDevlogEntries(project.id),
    getApprovedComments(project.id),
  ]);

  return (
    <>
      <script type="application/ld+json" dangerouslySetInnerHTML={{ __html: JSON.str
      <Navbar settings={...} />
      <ProjectHeader project={project} />
      <ProjectTabs project={project} entries={devlogEntries} comments={comments} />
    </>
  );
}
```

ProjectHeader: Cover image (full-width hero), title, tech stack badges, links row (GitHub, LinkedIn, Live Demo), Loom embed.

✓ `/projects/test-project` renders. OG metadata is correct. Structured data is in the HTML.

TASK 4.10 — Overview Tab

Time: 30 min

File: `components/public/ProjectPage/OverviewTab.tsx`

- Full description rendered via `MarkdownRenderer`
- Tech stack grouped by category (Language: ..., Framework: ..., etc.)
- Stats row: GitHub stars, commits, start date, status badge

✓ Markdown renders correctly with code syntax highlighting.

TASK 4.11 — Devlog Tab

Time: 2 hours

Files:

- `components/public/ProjectPage/DevlogTab.tsx`
- `components/public/ProjectPage/DevlogEntryCard.tsx`
- `components/public/ProjectPage/EditHistoryModal.tsx`

DevlogEntryCard renders each entry type with distinct visual treatment per the PRD:

- `milestone`: large card with colored date badge
- `issue`: red left-border accent
- `fix`: gold/green star accent, links back to paired issue if `linkedEntryId` exists
- `architecture`: blue accent
- `reflection`: italic styling
- `daily_log`: compact, neutral

Each card shows:

- Type icon + label
- Entry date (formatted)
- Title
- Markdown content (via `MarkdownRenderer`)
- "Last edited [date]" if `updatedAt !== createdAt`
- "View edit history" link if edited (fetches versions on click)

Issue ↔ Fix linking:

- On issue cards: if `linkedEntryId` is set → "See how this was fixed →" link that scrolls to the fix card
- On fix cards: "← This fixed: [issue title]" at the top
- Use DOM `id` attributes on cards (`id={entry.id}`) for scroll targets

EditHistoryModal:

- Opens when "View edit history" clicked
- Fetches versions via `getDevlogVersions` Server Action on open
- Lists versions: "Version 3 (current)", "Version 2 — saved May 12, 2025", etc.
- Clicking a version shows full Markdown content in a read-only viewer

Lazy loading:

- First 10 entries pass from Server Component
- "Load more" button fetches next 10 via client-side Firestore query

✅ Devlog tab renders all entries with correct styling. Issue/fix links work. Edit history modal opens and shows snapshots. Load more works.

TASK 4.12 — Discussion Tab

Time: 60 min

File: `components/public/ProjectPage/DiscussionTab.tsx`

- Comment list: approved comments sorted newest first
- Comment form: name + message (React Hook Form + Zod, 20-1000 chars)
- On submit: call `submitComment` → show "Submitted for review" toast
- Comments display: name, date, content
- Empty state: "No comments yet. Be the first to share your thoughts."

✅ Comment form submits. New comment appears in Firestore as pending. Admin can approve it. Approved comment appears in public view (after ISR revalidation or page refresh).

TASK 4.13 — Custom 404 Page

Time: 30 min

File: `app/not-found.tsx`

Creative 404 with your brand styling. Suggestions:

- "404: This route doesn't exist in production either."
- Navigation links back to home and projects
- Subtle animation

✓ Navigating to `/nonexistent-page` shows custom 404.

TASK 4.14 — Footer Component

Time: 20 min

File: `components/public/Footer.tsx`

Simple: name, copyright, social links (GitHub, LinkedIn, Twitter), "Built with Next.js + Firebase."

✓ Footer renders at bottom of all public pages.

PHASE 1E: Admin CMS (~18 hours)

TASK 5.1 — Admin Dashboard

Time: 45 min

File: `app/admin/dashboard/page.tsx`

Server Component (calls `verifyAdminSession`). Fetches:

- Published project count
- Projects by tier breakdown
- Pending comment count
- Unread message count
- Total devlog entry count
- Last devlog entry timestamp

Display as stat cards with icons. Use Firestore `onSnapshot` for real-time count updates (pass initial data from server, hydrate with real-time listener on client).

✓ Dashboard shows all stats. Approving a comment decrements the badge in real time.

TASK 5.2 — Project List (Admin)

Time: 2 hours

File: `app/admin/projects/page.tsx`, `components/admin/ProjectTable.tsx`

This is the most complex admin page. Build it in layers:

Layer 1 — Basic table:

- Columns: Cover image thumbnail (40x40), Title, Tier (dropdown), Order (number input), Status toggle, Edit link, Delete button
- Fetch all projects via `getAllProjectsAdmin()`

Layer 2 — Inline tier change:

- Tier dropdown `onChange` → call `changeProjectTier` Server Action
- Optimistic UI: update local state immediately, rollback if action fails
- Toast on success/fail

Layer 3 — Promote/Demote buttons:

- ▲ button: calls `changeProjectTier(id, tier + 1)`. Disabled if already tier 3.
- ▼ button: calls `changeProjectTier(id, tier - 1)`. Disabled if already 0.

Layer 4 — Filter + Search:

- Filter bar: Tier select (All/0/1/2/3), Status select (All/Published/Draft)
- Search input: client-side filter on title
- Filter state in URL params (survives page refresh)

Layer 5 — Pagination:

- 20 rows per page
- Previous/Next buttons
- "Showing 1-20 of 47" indicator

✅ All 5 layers work together. Changing a project tier updates immediately in the table and on the homepage after ISR revalidation.

TASK 5.3 — Project Create/Edit Form

Time: 3 hours

Files: `app/admin/projects/new/page.tsx`, `app/admin/projects/[id]/edit/page.tsx`,
`components/admin/ProjectForm.tsx`

Build `ProjectForm` as a single form component that handles both create and edit (receives optional `project` prop; if provided, pre-fills all fields).

Form sections (match PRD Epic 3, E3-S2 exactly):

1. **Identity:** Title input → auto-generates slug (show preview, allow override) → short description textarea → full description Markdown editor (`@uiw/react-md-editor`) → cover image upload → Published toggle
2. **Display:** Tier select → Order input → Featured toggle
3. **Links:** GitHub URL → Loom Demo ID (show preview iframe when filled) → Loom Arch ID (optional, same) → LinkedIn URL → Live Demo URL
4. **Tech Stack:** Tag input with category select per tag. Add tags with Enter key. Render as removable badge chips. Pre-built category colors.
5. **Dates:** Start date picker → End date picker OR "Ongoing" toggle → Status select (Active/Completed/Archived)

Validation: All fields validated with Zod on submit.

Cover image upload (`components/admin/FileUploadInput.tsx`):

- `<input type="file" accept="image/*">`
- On change: upload to Firebase Storage via client SDK → `ref(storage, /images/projects/${projectId}-cover.webp)`
- Show upload progress bar
- On complete: set `coverImageUrl` field value via `setValue` (React Hook Form)
- Show image preview

On form submit: Call `upsertProject` Server Action → redirect to project edit page (with success toast).

✅ Create a new project via the form. It appears in the project list and on the public homepage (after ISR). Edit the project → changes reflect on the public page.

TASK 5.4 — Devlog Entry Management (Inside Project Edit)

Time: 2 hours

Files: `components/admin/DevlogEntryList.tsx`, `components/admin/DevlogEntryForm.tsx`

The project edit form has a "Devlog" tab. Inside it:

DevlogEntryList:

- Lists all entries for this project (newest first, admin view)

- Each row: type badge, title, entry date, published toggle, Edit button, Delete button
- "New Entry" button at top

DevlogEntryForm (modal or inline panel):

- Type select (dropdown with icons)
- Entry title
- Entry date (date picker, defaults to today)
- Content (Markdown editor with full preview)
- Published toggle
- "Link to Issue" select (appears when type = 'fix' — searches existing issue entries)
- Save button → calls `upsertDevlogEntry`

Auto-versioning is invisible: The admin just clicks "Save." Versions are stored automatically by the Server Action. No UI needed for the write side.

Version history view:

- Each entry row has a "History" icon button
- Clicking it opens a modal calling `getDevlogVersions` → shows snapshot list

✅ Write 3 versions of a devlog entry. `getDevlogVersions` returns 2 snapshots (original + first edit; current version is not in versions collection).

TASK 5.5 — Devlog Quick-Access (Admin Sidebar)

Time: 45 min

File: `app/admin/projects/devlog/page.tsx`

Standalone devlog writing page accessible from sidebar. Two-step flow:

1. Select project from dropdown (fetches all projects)
2. Opens DevlogEntryForm pre-configured for that project

Useful for the daily writing workflow without navigating to the full project edit form.

✅ Admin can write a devlog entry from the sidebar in 2 clicks.

TASK 5.6 — Comment Moderation Queue

Time: 60 min

File: `app/admin/comments/page.tsx`, `components/admin/CommentModerationTable.tsx`

- Fetches all pending comments via `getAllPendingComments()`
- Table columns: Author, Comment excerpt (first 100 chars), Project (linked to public page), Date, Approve (✓), Delete (X)
- Bulk select checkboxes → "Approve All Selected" + "Delete All Selected" buttons
- After approve/delete: row removes with fade-out animation, count badge updates

✓ Approve a comment from the queue. It appears in the public Discussion tab. Delete a comment → gone from both queue and public view.

TASK 5.7 — Timeline CRUD

Time: 60 min

File: `app/admin/timeline/page.tsx`

- List of timeline events with Edit/Delete per row
- "Add Event" button opens a form modal
- Form fields: type, title, organization, location, dateRange, startDate, endDate, description (textarea, one bullet per line — split on save), logoUrl
- Logo upload via `FileUploadInput`

✓ Add a timeline event → appears on public homepage timeline section.

TASK 5.8 — Contact Inbox

Time: 45 min

File: `app/admin/inbox/page.tsx`

- Real-time list of contact submissions (Firestore `onSnapshot()`)
- Status filter: All / Unread / Read / Archived
- Click a submission → expand to full message, mark as read, archive
- Badge count in sidebar updates in real time

✓ Submit contact form → immediately appears in admin inbox as "Unread."

TASK 5.9 — CV Upload

Time: 30 min

File: `app/admin/cv/page.tsx`

- ✅ Upload a new PDF. The "Download CV" button on the public homepage serves the new file.
-

TASK 5.10 — Settings Page

Time: 45 min

File: `app/admin/settings/page.tsx`

Form pre-filled with current `settings/global` document. Fields:

- Display name, tagline, sub-headline
- Avatar upload
- GitHub URL, LinkedIn URL (global fallback), Twitter URL
- Email
- SEO title suffix, meta description

On save: calls `updateGlobalSettings` → revalidates homepage.

- ✅ Change the tagline → appears on public homepage within 5 minutes (ISR window).
-

PHASE 1F: SEO, Performance, & CI/CD (~8 hours)

TASK 6.1 — Sitemap & Robots

Time: 30 min

Files: `app/sitemap.ts`, `app/robots.ts`

Implement sitemap from TRD Section 18.3. robots.ts:

```
typescript
```

```
export default function robots() {
  return {
    rules: { userAgent: '*', allow: '/', disallow: '/admin/' },
    sitemap: 'https://yourdomain.com/sitemap.xml',
  };
}
```

✓ `/sitemap.xml` includes all published project slugs. `/robots.txt` disallows `/admin/`.

TASK 6.2 — Dynamic OG Images

Time: 45 min

File: `app/projects/[slug]/opengraph-image.tsx`

typescript

```
import { ImageResponse } from 'next/og';
import { getProjectBySlug } from '@lib/actions/projects';

export const size = { width: 1200, height: 630 };
export const contentType = 'image/png';

export default async function Image({ params }: { params: { slug: string } }) {
  const project = await getProjectBySlug(params.slug);

  return new ImageResponse(
    (
      <div style={{ background: '#0f172a', width: '100%', height: '100%', display: 'flex', align-items: center, justify-content: center; }}>
        <div style={{ color: '#0ea5e9', fontSize: 24, marginBottom: 16 }}>AI Engineer</div>
        <div style={{ color: 'white', fontSize: 64, fontWeight: 700, lineHeight: 1.1 }}>{project?.name}</div>
        <div style={{ color: '#94a3b8', fontSize: 28, marginTop: 24 }}>{project?.slug}</div>
      </div>
    ),
    size,
  );
}
```

✓ Sharing a project URL on LinkedIn shows the dynamic OG image.

TASK 6.3 — Performance Audit Pass

Time: 2 hours

Run Lighthouse locally:

```
bash

npm run build && npm run start
# In another terminal:
npx lighthouse http://localhost:3000 --view
```

Common issues and fixes:

Issue	Fix
LCP > 2.5s	Add <code>priority</code> to hero image. Check font loading.
CLS > 0	Add explicit <code>width</code> + <code>height</code> to all <code><Image></code> components. Add <code>aspect-ratio</code> CSS to Loom iframes.
Accessibility < 100	Every <code></code> needs <code>alt</code> . Every button needs accessible label. Color contrast ratios must be $\geq 4.5:1$.
SEO < 100	Check <code><meta name="description"></code> is present on all pages. Sitemap is valid XML.

Key Tailwind patterns for CLS-free Loom embeds:

```
html

<div class="relative w-full" style="padding-bottom: 56.25%">
  <iframe class="absolute inset-0 w-full h-full" src="https://www.loom.com/embed/..."
</div>
```

✓ All four Lighthouse categories score ≥ 95 on homepage and test project page.

TASK 6.4 — Lighthouse CI Config

Time: 20 min

File: `.lighthouserc.json`

Copy exact config from TRD Section 17.3.

✓ File exists with all assertions configured.

TASK 6.5 — GitHub Actions CI Workflow

Time: 45 min

File: `.github/workflows/ci.yml`

Copy exact workflow from TRD Section 17.3.

Required GitHub Secrets (add in GitHub → Settings → Secrets → Actions):

- `VERCEL_TOKEN`: from Vercel → Account Settings → Tokens
- `VERCEL_ORG_ID`: from Vercel project settings
- `VERCEL_PROJECT_ID`: from Vercel project settings

✓ Open a PR to `dev`. GitHub Actions runs CI. Lighthouse audit passes. PR can be merged.

TASK 6.6 — Production Deploy Workflow

Time: 20 min

File: `.github/workflows/deploy.yml`

Copy exact workflow from TRD Section 17.4.

Add environment variables to Vercel: Go to Vercel → Project → Settings → Environment Variables → add all variables from `.env.local`.

✓ Push to `main` → Vercel deploys production automatically. Site is live at your Vercel domain.

TASK 6.7 — First Production Content

Time: 1 hour

With the production site live:

1. Login to `/admin/login` on production
2. Create your 3 real Tier 1 projects with real GitHub repos, Loom videos, and descriptions
3. Create your timeline events
4. Upload your real CV
5. Update settings with real social links

✓ Production site shows real content. All Lighthouse scores ≥ 95 . CV download works.

PHASE 1G: Final Polish (~4 hours)

TASK 7.1 — Animations Pass

Time: 60 min

Add Framer Motion scroll-reveal to:

- Timeline events (staggered reveal)
- Tier 1 project cards (stagger 0.1s each)
- Hero section elements
- Contact form fields

Ensure `prefers-reduced-motion` is respected:

```
typescript
```

```
const prefersReducedMotion = window.matchMedia('(prefers-reduced-motion: reduce)').media
```

In Framer Motion: use `useReducedMotion()` hook and conditionally skip animations.

✅ Animations work. `prefers-reduced-motion` disables them gracefully.

TASK 7.2 — Error Boundaries & Loading States

Time: 45 min

- `app/loading.tsx`: Full-page skeleton for route transitions
- `app/projects/[slug]/loading.tsx`: Project page skeleton
- `app/error.tsx`: Error boundary with "Something went wrong" + refresh button
- Add `<Suspense>` boundaries around all async data-fetching components

✅ Navigating between pages shows skeletons. Simulating a Firestore error shows the error boundary.

TASK 7.3 — Accessibility Audit

Time: 45 min

Run axe DevTools or use Lighthouse accessibility panel. Fix everything to reach 100:

- Focus management in modals (trap focus, restore on close)
- Skip navigation link at top of page (`<a href="#main-content">Skip to content</a>`)
- All form inputs have associated labels
- Color contrast: use `tailwind-accessible-colors` or manually check
- ARIA landmarks: `<nav>`, `<main>`, `<footer>`, `<section aria-labelledby="...">`

✓ Lighthouse Accessibility: 100/100.

TASK 7.4 — Mobile Responsiveness Audit

Time: 30 min

Test every page at 375px, 768px, 1280px. Common issues:

- Tier 1 cards: stack vertically on mobile, side-by-side on desktop
- Admin table: horizontal scroll on mobile (use `overflow-x-auto` wrapper)
- Markdown content: code blocks need `overflow-x-auto`
- Navigation: hamburger menu opens/closes correctly

✓ All pages usable on iPhone SE (375px) screen width.

TASK 7.5 — Phase 1 Completion Checklist

Before tagging v1.0.0:

- ☐ All public pages render with real content
- ☐ All admin operations work end-to-end
- ☐ Lighthouse scores: ≥ 95 all four categories (on live site, not localhost)
- ☐ CV download works
- ☐ Contact form delivers email
- ☐ Comment submission and moderation works
- ☐ Devlog versioning works
- ☐ Dark mode works with no FOUC
- ☐ Mobile navigation works
- ☐ CI pipeline blocks on Lighthouse regression
- ☐ No `.env.local` values committed to Git
- ☐ Admin UID not visible in public code

```
git tag v1.0.0
git push origin v1.0.0
```

PHASE 2: AI Chatbot Integration (~36 hours)

Begin Phase 2 only after Phase 1 is deployed, stable, and live for at least 1-2 weeks. Phase 2 requires paid API keys (OpenAI + Pinecone) — factor this into timing.

TASK 8.1 — Additional Dependencies

Time: 15 min

```
bash

npm install openai @pinecone-database/pinecone ai
```

Add to `.env.local`:

```
OPENAI_API_KEY=
PINECONE_API_KEY=
PINECONE_INDEX_NAME=portfolio-rag
PINECONE_ENVIRONMENT=
```

✓ Dependencies install. No version conflicts.

TASK 8.2 — Pinecone Index Setup

Time: 30 min

1. Create account at pinecone.io
2. Create index: name = `portfolio-rag`, dimensions = `1536` (text-embedding-3-small output), metric = `cosine`
3. Free tier: 1 index, 100k vectors — more than enough

✓ Index created and accessible via Pinecone dashboard.

TASK 8.3 — Embedding & Ingestion Pipeline

Time: 3 hours

typescript

```

// lib/rag/ingest.ts - server-only
import OpenAI from 'openai';
import { Pinecone } from '@pinecone-database/pinecone';

const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });
const pinecone = new Pinecone({ apiKey: process.env.PINECONE_API_KEY! });

export async function embedText(text: string): Promise<number[]> {
  const res = await openai.embeddings.create({
    model: 'text-embedding-3-small',
    input: text,
  });
  return res.data[0].embedding;
}

export interface IngestChunk {
  id: string; // Unique - determines upsert behavior (same id = overwrite)
  text: string;
  metadata: {
    source: 'cv' | 'project_description' | 'devlog_entry' | 'readme';
    projectId?: string;
    projectName?: string;
    entryId?: string;
    entryType?: string;
    entryTitle?: string;
  };
}

export async function ingestChunks(chunks: IngestChunk[]) {
  const index = pinecone.index(process.env.PINECONE_INDEX_NAME!);

  // Embed all chunks in parallel (batch to avoid rate limits)
  const batchSize = 20;
  for (let i = 0; i < chunks.length; i += batchSize) {
    const batch = chunks.slice(i, i + batchSize);
    const embeddings = await Promise.all(batch.map(c => embedText(c.text)));

    await index.upsert(
      batch.map((chunk, j) => ({
        id: chunk.id,
        values: embeddings[j],
        metadata: { text: chunk.text, ...chunk.metadata },
      }))
    )
  }
}

```

```

    );
  }
}

export async function deleteChunk(id: string) {
  const index = pinecone.index(process.env.PINECONE_INDEX_NAME!);
  await index.deleteOne(id);
}

```

✅ `ingestChunks([ { id: 'test-1', text: 'Hello world', metadata: { source: 'cv' } } ])` writes to Pinecone.

TASK 8.4 — Trigger Ingestion from Server Actions

Time: 45 min

Modify existing Server Actions to trigger re-ingestion:

In `upsertDevlogEntry` (when `published: true` is being set or was already true):

```

typescript

// After saving to Firestore:
if (data.published) {
  await ingestChunks([
    id: `devlog-${projectId}-${entryId}`,
    text: `${data.title}\n\n${data.content}`,
    metadata: {
      source: 'devlog_entry',
      projectId,
      projectTitle: project.title,
      entryId,
      entryType: data.type,
      entryTitle: data.title,
    }
  ]);
}

// If unpublishing: deleteChunk(`devlog-${projectId}-${entryId}`)

```

In `upsertProject` (when saved/updated):

```

typescript

```

```
await ingestChunks([{\n  id: `project-${projectId}`,\n  text: `${data.title}\\n\\n${data.shortDescription}\\n\\n${data.fullDescription}`,\n  metadata: { source: 'project_description', projectId, projectName: data.title }\n}]);
```

In CV upload action:

```
typescript\n\n// After uploading PDF to Firebase Storage:\n// Download the PDF, extract text, chunk by section, ingest\nconst pdfText = await extractTextFromPdf(cvBuffer);\nconst sections = chunkBySections(pdfText);\nawait ingestChunks(sections.map((s, i) => ({\n  id: `cv-section-${i}`,\n  text: s,\n  metadata: { source: 'cv' }\n})));
```

For PDF text extraction: `npm install pdf-parse`

✅ Saving a devlog entry creates a vector in Pinecone with the correct metadata.

TASK 8.5 — Chat Route Handler (Streaming)

Time: 3 hours

File: `app/api/chat/route.ts`

```
typescript
```

```

import { NextRequest } from 'next/server';
import OpenAI from 'openai';
import { Pinecone } from '@pinecone-database/pinecone';
import { embedText } from '@lib/rag/ingest';

const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });
const pinecone = new Pinecone({ apiKey: process.env.PINECONE_API_KEY! });

export async function POST(req: NextRequest) {
  const { message, history } = await req.json();

  if (!message || typeof message !== 'string' || message.length > 500) {
    return new Response('Invalid message', { status: 400 });
  }

  // 1. Embed the user query
  const queryEmbedding = await embedText(message);

  // 2. Vector search – top 5 most relevant chunks
  const index = pinecone.index(process.env.PINECONE_INDEX_NAME!);
  const results = await index.query({
    vector: queryEmbedding,
    topK: 5,
    includeMetadata: true,
  });

  const context = results.matches
    .filter(m => (m.score ?? 0) > 0.7) // Relevance threshold
    .map(m => m.metadata?.text as string)
    .join('\n\n---\n\n');

  // 3. Build augmented messages
  const systemPrompt = `You are a helpful assistant for [Your Name]'s engineering portfolio. Answer questions about their projects, skills, and experience based on the context provided. Be concise, direct, and friendly. If the context doesn't contain the answer, say so.`;

  Context from portfolio:
  ${context || 'No relevant context found.'}`;

  // 4. Stream response using the AI SDK
  const stream = await openai.chat.completions.create({
    model: 'gpt-4o-mini',
    stream: true,
  });

```

```

    messages: [
      { role: 'system', content: systemPrompt },
      ...history.slice(-6), // Last 3 turns for context
      { role: 'user', content: message },
    ],
    max_tokens: 500,
  });

// 5. Return as SSE stream
const encoder = new TextEncoder();
const readable = new ReadableStream({
  async start(controller) {
    for await (const chunk of stream) {
      const text = chunk.choices[0]?.delta?.content ?? '';
      if (text) {
        controller.enqueue(encoder.encode(`data: ${JSON.stringify({ text })}\n\n`));
      }
    }
    controller.enqueue(encoder.encode('data: [DONE]\n\n'));
    controller.close();
  },
});

return new Response(readable, {
  headers: {
    'Content-Type': 'text/event-stream',
    'Cache-Control': 'no-cache',
    Connection: 'keep-alive',
  },
});
}

```

✓ POST to `/api/chat` with `{ message: "What projects have you built?", history: [] }` returns a streaming response with relevant content from your devlog entries.

TASK 8.6 — Chat Widget UI

Time: 2 hours

File: `components/public/ChatWidget/ChatWidget.tsx`

- Floating button: bottom-right, fixed position, z-50
- Click → panel opens (380×520px, full screen on mobile)

- Panel header: "Ask about my work" + close button
- Message thread: user messages right-aligned, assistant left-aligned
- Typing indicator while streaming (animated dots)
- Suggested prompts (shown when thread is empty):
 - "What's your strongest AI project?"
 - "What's your experience with RAG?"
 - "Are you available for remote work?"
- Input field + send button
- SSE streaming: read response from `/api/chat` and append tokens as they arrive
- Session message limit: 15 (tracked in `sessionStorage`)

SSE client implementation:

```
typescript
```

```

const sendMessage = async (text: string) => {
  addMessage({ role: 'user', content: text });
  setLoading(true);
  let buffer = '';

  const res = await fetch('/api/chat', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ message: text, history: messages }),
  });

  const reader = res.body!.getReader();
  const decoder = new TextDecoder();

  addMessage({ role: 'assistant', content: '' }); // Placeholder

  while (true) {
    const { done, value } = await reader.read();
    if (done) break;
    const lines = decoder.decode(value).split('\n').filter(l => l.startsWith('data:'));
    for (const line of lines) {
      const data = line.slice(6);
      if (data === '[DONE]') break;
      const { text } = JSON.parse(data);
      buffer += text;
      updateLastMessage(buffer); // Update the placeholder message
    }
  }
  setLoading(false);
};

```

✓ Chat widget opens. Sending "What projects have you built?" returns a relevant streamed response. Message counter shows remaining messages.

TASK 8.7 — Admin Knowledge Base Page

Time: 45 min

File: `app/admin/knowledge/page.tsx`

Shows what's in the Pinecone index:

- List all vector IDs by prefix: CV sections, project descriptions, devlog entries

- "Re-index All Projects" button (triggers manual re-ingestion of all published projects)
- "Re-index CV" button
- Last indexed timestamp per source

✓ Admin can view and force-refresh the knowledge base.

TASK 8.8 — Phase 2 Testing

Time: 1 hour

Test these queries against the chat widget:

1. "What's the most complex project you've built?" → should reference NEXUS/JurisGuard
2. "What's your experience with LangChain?" → should pull from project descriptions
3. "How did you fix the GPU loading issue?" → should pull from devlog fix entries
4. "Tell me about yourself" → should summarize from CV sections
5. "Are you available for remote work?" → should answer from CV/settings

✓ All 5 queries return relevant, accurate responses drawn from the vectorized portfolio content.

APPENDIX A: Implementation Order Summary

Week 1 (Pre-Phase + 1A): Setup, types, Firebase, auth system

Week 2 (1B + 1C): Admin auth, all Server Actions

Week 3 (1D): All public pages

Week 4 (1E): All admin CMS pages

Week 5 (1F + 1G): SEO, performance, CI/CD, polish

Week 6-8 (Phase 2): RAG chatbot integration

APPENDIX B: Decision Log

Decision	Rationale
Build types FIRST (Task 1.1)	Prevents type mismatches discovered late; contracts are clear from the start
Build Server Actions BEFORE UI (Phase 1C before 1D)	UI just calls actions; no prop-drilling guessing

Decision	Rationale
Admin auth BEFORE admin UI (Phase 1B before 1E)	Never build a page that doesn't have its security wrapper
Seed test data before building pages (Task 1.7)	Pages need real Firestore data to render; don't work with empty screens
ISR at 300s (not 60s)	Reduces Firestore read costs significantly at the cost of max 5min propagation delay
Auto-versioning on SERVER side only	Client can never fake a version; Admin SDK bypasses security rules cleanly
Phase 2 after stable Phase 1	RAG requires ongoing content to be useful; indexing an empty devlog returns poor results

APPENDIX C: Debugging Reference

Symptom	Likely Cause	Fix
Firestore reads return empty	Security rules blocking	Check <code>isAdmin()</code> function — verify <code>YOUR_ADMIN_UID</code> is replaced
Admin session cookie not set	HTTPS required in production for <code>secure: true</code>	Use Vercel preview URL (HTTPS) for testing
<code>FIREBASE_ADMIN_PRIVATE_KEY</code> errors	Escaped newlines in env var	Wrap in double quotes; replace <code>\n</code> with actual newlines in the <code>admin.ts</code> init
Lighthouse LCP > 2.5s	Loom iframe loading on visible page	Add <code>loading="lazy"</code> to all iframes not in initial viewport
ISR not revalidating	<code>revalidatePath</code> called with wrong path	Ensure path matches EXACTLY (e.g., <code>/projects/test-project</code> not <code>/projects/[slug]</code>)
Devlog versions not created	Security rules block Admin SDK writes to	Verify the action uses <code>adminDb</code> not the client <code>db</code>

Symptom	Likely Cause	Fix
	versions	
Chat returns wrong answers	Low relevance threshold (< 0.7)	Lower to 0.6, or re-index with better chunking

End of Implementation Plan — v1.0 Total estimated hours: ~116 (Phase 1: ~80h, Phase 2: ~36h)